

Practically Groovy: Unit test your Java code faster with Groovy

Skill Level: Introductory

[Andrew Glover](mailto:andrew@thirstyhead.com) (andrew@thirstyhead.com)
Co-Founder
ThirstyHead.com

[Scott Davis](mailto:scott@thirstyhead.com) (scott@thirstyhead.com)
Founder
ThirstyHead.com

09 Nov 2004

Not long ago, developerWorks contributor Andrew Glover penned an article introducing Groovy, a new proposed standard language for the Java platform, as part of our *alt.lang.jre* series. Reader response was fantastic, so we've decided to launch this column to offer a practical guide to using this hot new technology. This first installment introduces a simple strategy for unit testing Java code with Groovy and JUnit.

I'll start with a confession: I'm a unit testing addict. In fact, I just can't write enough unit tests. If I'm developing for long stretches of time *without* having written corresponding unit tests, I get the jitters. Unit tests give me the confidence that my code works and that I can change it, at a moment's notice, without the fear of it breaking.

Furthermore, as an addict, I tend to write a plethora of test cases. My high, however, isn't from *writing* the test cases; it's in *seeing* their results. Consequently, if I can write the tests in a rapid manner, I can view their results quicker. That way I feel better. Quicker.

Of late, I've been looking to Groovy to appease my unit testing addiction, and so far I'm impressed. The agility this new language brings to unit testing is quite exciting and worthy of some serious exploration. In this article, the first in a new series introducing the practical aspects of Groovy, I'll introduce you to the pleasures of unit

testing with Groovy. I'll start with an overview of Groovy's unique contributions to development on the Java platform, then move on to discuss the particulars of unit testing with Groovy and JUnit, with special emphasis on Groovy's extension of JUnit's `TestCase` class. I'll conclude with a working example that shows you, first hand, how to integrate these *groovy* features with Eclipse and Maven.

No more Java purism!

Before I launch into the practical aspects of unit testing with Groovy, I think it's important to talk about the more general issue of its place in your development toolbox. The fact is, Groovy isn't the only scripting language that runs on the Java Runtime Environment (JRE), it's just the only one that has been proposed as a standard language for the Java platform. As some of you will have learned from the *alt.lang.jre* series (see [Resources](#)), there are myriad options when it comes to scripting for the Java platform, most of them presenting highly agile environments for rapid application development.

Despite this abundance of choices, many developers choose to stick with their favorite and most-familiar paradigm: the Java language. While Java programming is a fine choice for most situations, there is one very important shortcoming to wearing Java-only blinders. As a wise person once put it: *If the only tool you have is a hammer, you tend to see every problem as a nail*. I think there's a lot of truth to this saying that is applicable to software development.

Just as I hope to convince you with this series that the Java language is not and should not be your only choice for developing applications, it's also true that scripting languages make sense in some scenarios and not in others. What separates the professional from the tyro is knowing when to *apply* the power of scripting and when to *eschew* it.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

For example, scripting is typically not such a good fit for high-performance, transaction-intensive, enterprise-wide applications; for these cases your best bet *could* be a normal J2EE stack. On the other hand, scripting -- and particularly scripting with Groovy -- can make a lot of sense when it comes to rapid prototyping of small, highly specific applications that are not performance intensive, such as configuration systems and/or build systems. It's also a near-perfect fit for reporting

applications and, most importantly, unit testing.

Why unit test with Groovy?

What makes Groovy particularly appealing with respect to other scripting platforms is its seamless integration with the Java platform. Because it's based on the Java language (unlike other alternate languages for the JRE, which tend to be based on earlier predecessors), Groovy presents an incredibly short learning curve for the Java developer. And once that learning curve has straightened out, Groovy can offer an unparalleled rapid development platform.

The secret to Groovy's success, in this regard, is its syntax, which *is* Java syntax, but with far fewer rules. For example, Groovy doesn't require semicolons, and it makes variable types and access modifiers optional. Moreover, Groovy makes use of the standard Java libraries you're already familiar with, including `Collections` and `File/IO`. And, finally, you can utilize any Java library from within Groovy, including `JUnit`.

The fact is, Groovy's relaxed Java-like syntax, its reuse of standard Java libraries, and its rapid build-and-run cycle make it an ideal candidate for rapidly developing unit tests. But don't just take my word for it; let's see it in code!

JUnit and Groovy

Unit testing Java code in Groovy couldn't be easier, and there are many options for getting started. The most straightforward choice is to stick with the industry standard, `JUnit`. The simplicity and power of `JUnit` are unrivaled, its ubiquity as a helpful Java development tool is unparalleled, and there's nothing stopping the combination of `JUnit` and Groovy, so why reinvent the wheel? In fact, once you've seen `JUnit` and Groovy together in action, I'll bet you'll never turn back! The key thing to remember here is that you can do all the same things with `JUnit` in Groovy that you can do in the Java language; albeit with far fewer keystrokes.

Getting started

Once you've downloaded `JUnit` and Groovy (see [Resources](#)) you can proceed in one of two ways. The first option is to write normal `JUnit` test cases just as you've been doing all along by extending `JUnit`'s commendable `TestCase`. The second option is to apply Groovy's nifty `GroovyTestCase` extension, which will consequently extend `JUnit`'s `TestCase`. The first option is your quickest path to success with maximum *Java-esque* familiarity. The second option, on the other hand, pushes you into the Groovy world with maximum agility.

To get started, imagine a Java object that applies a filter to a given `string` and

returns a boolean value corresponding to a match. This filter could be a simple string operation such as `indexOf()` or, more powerfully, it could be a regular expression. The desired filter is set at runtime via a `setFilter()` method and the `apply()` method takes the string to be filtered. Listing 1 shows this example Filter interface in plain Java code:

Listing 1. A simple Java Filter interface

```
public interface Filter {  
    void setFilter(String fltr);  
    boolean applyFilter(String value);  
}
```

The idea is to use this feature to filter out desired or undesired package names from a large list. Consequently, I've created two implementations: `RegexPackageFilter` and `SimplePackageFilter`.

Combining the power and simplicity of Groovy and JUnit yields the elegant test suite shown in Listing 2:

Listing 2. A Groovy RegexFilterTest using JUnit

```
import junit.framework.TestCase  
import com.vanward.sedona.frmwrk.filter.impl.RegexPackageFilter  
  
class RegexFilterTest extends TestCase {  
    void testSimpleRegex() {  
        def fltr = new RegexPackageFilter()  
        fltr.setFilter("java.*")  
        def val = fltr.applyFilter("java.lang.String")  
        assertEquals("value should be true", true, val)  
    }  
}
```

The code in Listing 2 should look familiar to you regardless of whether you're familiar with Groovy, because it's simply Java code without any semicolons, access modifiers, or variable types! The above JUnit test has one test case, `testSimpleRegex()`, which attempts to assert that the `RegexPackageFilter` has correctly found a match with `"java.lang.String"` using the regular expression `"java.*"`.

Groovy extends JUnit

Extending JUnit's `TestCase` class to add additional features is a common technique utilized in virtually every JUnit extension. The DbUnit framework (see [Resources](#)), for example, offers a handy `DatabaseTestCase` that makes managing the state of a database easier than ever, and the indispensable `MockStrutsTestCase` (from

the StrutsTestCase framework; see [Resources](#)) yields a virtual servlet container for executing struts code. Both of these powerful frameworks have elegantly extended JUnit to provide features otherwise not found in its core code; and now Groovy has gone and done it too!

Like StrutsTestCase and DbUnit, Groovy's extension of JUnit's TestCase brings some important new features to your developer toolbox. This particular extension lets you run test suites via the groovy command, and also offers up a host of new assert methods. These methods can be handy when it comes to asserting that a script correctly runs, asserting the lengths of various array types and the contents of various array types, and more.

Fun with GroovyTestCase

There's no better way to learn what GroovyTestCase can do for you than to see it in action. In Listing 3, I've written a new SimpleFilterTest, but this time I've extended GroovyTestCase to do it:

Listing 3. An actual GroovyTestCase

```
import groovy.util.GroovyTestCase
import com.vanward.sedona.frmwrk.filter.impl.SimplePackageFilter

class SimpleFilterTest extends GroovyTestCase {

    void testSimpleJavaPackage() {
        def fltr = new SimplePackageFilter()
        fltr.setFilter("java.")
        def val = fltr.applyFilter("java.lang.String")
        assertEquals("value should be true", true, val)
    }
}
```

Note that this test suite can be run via the command line without the main() method you would need in order to run Java-based JUnit test suite. In fact, if I wrote the above SimpleFilterTest in Java code, it would look something like the one shown in Listing 4:

Listing 4. The same test case in Java code

```
import junit.framework.TestCase;
import com.vanward.sedona.frmwrk.filter.Filter;
import com.vanward.sedona.frmwrk.filter.impl.SimplePackageFilter;

public class SimplePackageFilterTest extends TestCase {

    public void testSimpleRegex() {
        Filter fltr = new SimplePackageFilter();
        fltr.setFilter("java.");
        boolean val = fltr.applyFilter("java.lang.String");
        assertEquals("value should be true", true, val);
    }
}
```

```
    }  
    public static void main(String[] args) {  
        junit.textui.TestRunner.run(SimplePackageFilterTest.class);  
    }  
}
```

Testing with assertions

In addition to letting you run tests via the command line, `GroovyTestCase` gives you some particularly handy `assert` methods. `assertArrayEquals`, for example, asserts that two arrays are equal by checking their individual values and respective lengths. You can see Groovy assertions in action starting with the example in Listing 5, a nifty Java-based method that splits strings into arrays. (Note that I could have used Java 1.4's added string features to write the example class below. I used the Jakarta Commons `StringUtils` class to ensure backward-compatibility with Java 1.3.)

Listing 5. Defining a Java `StringSplitter` class

```
import org.apache.commons.lang.StringUtils;  
  
public class StringSplitter {  
    public static String[] split(final String input, final String separator){  
        return StringUtils.split(input, separator);  
    }  
}
```

Listing 6 shows how simple it is to test this class with the Groovy test suite and its corresponding `assertArrayEquals` method:

Listing 6. Using `assertArrayEquals` in a `GroovyTestCase`

```
import groovy.util.GroovyTestCase  
import com.vanward.resource.string.StringSplitter  
  
class StringSplitTest extends GroovyTestCase {  
    void testFullSplit() {  
        def splitAr = StringSplitter.split("groovy.util.GroovyTestCase", ".")  
        def expect = ["groovy", "util", "GroovyTestCase"].toArray()  
        assertArrayEquals(expect, splitAr)  
    }  
}
```

More ways to play

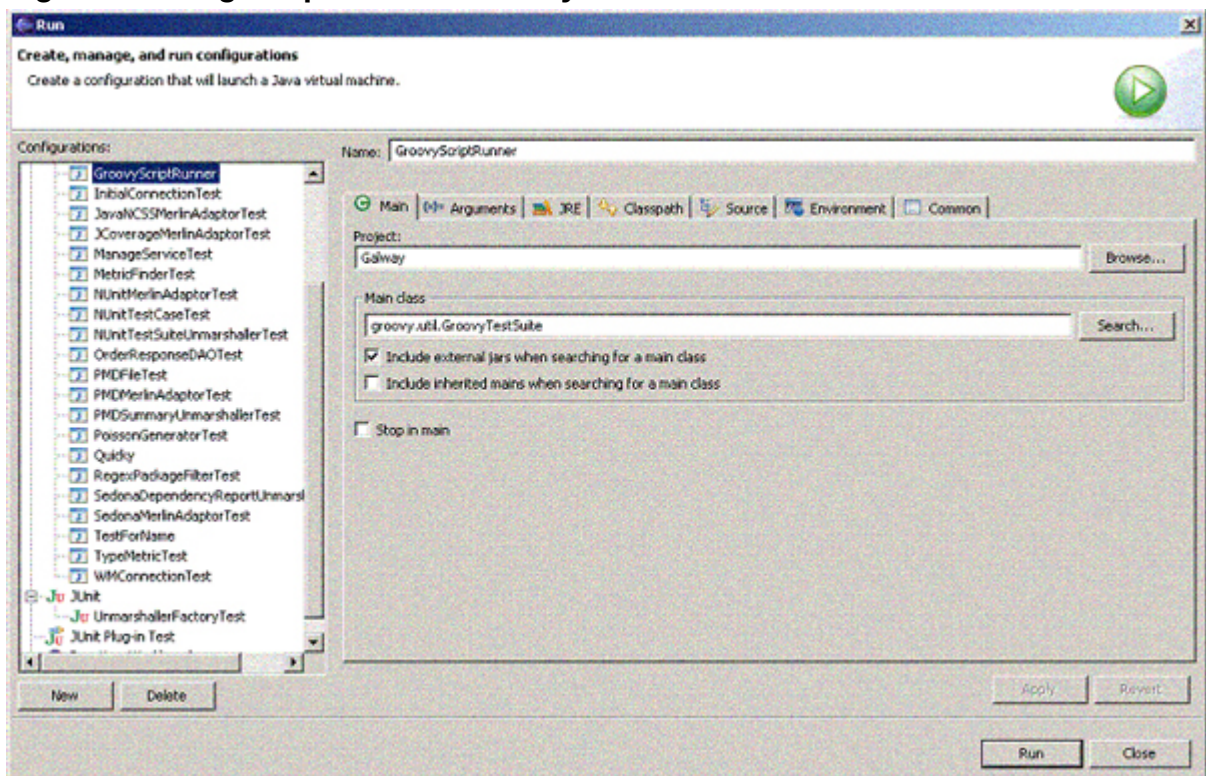
Groovy lets you run tests singly or in batches. With the `GroovyTestCase` extension, running a single test is effortless. Simply run the `groovy` command followed by the desired test suite and you're good to go, as shown in Listing 7:

Listing 7. Running a GroovyTestCase via the groovy command

```
./groovy test/com/vanward/sedona/frmwk/filter/impl/SimpleFilterTest.groovy
Time: 0.047
OK (1 test)
```

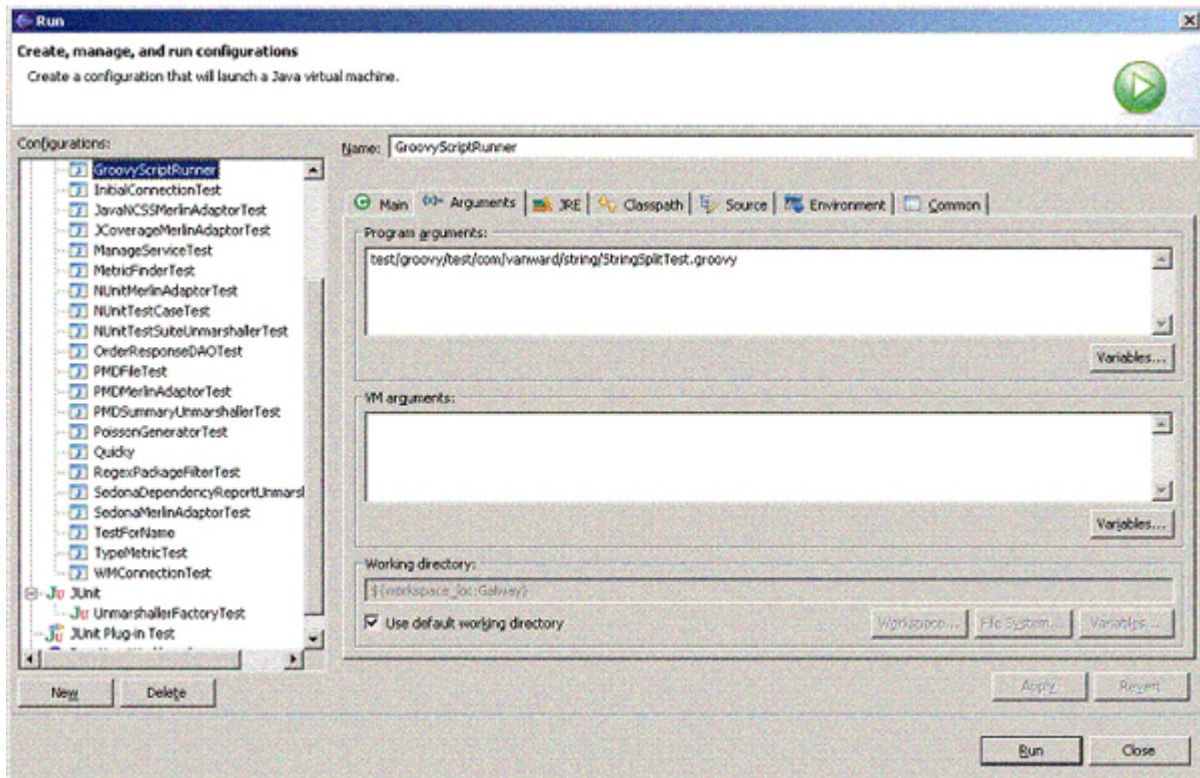
Groovy also offers a standard JUnit test suite called `GroovyTestSuite`. Simply run this test suite and pass in the path to your script. This test suite will run your script much like the `groovy` command. The nice thing about this technique is that it lets you run scripts in an IDE. For example, in Eclipse, I simply create a new run configuration for the example project (being sure to check "Include external jars when searching for a main class,") and then locate the main class `groovy.util.GroovyTestSuite`, as shown in Figure 1:

Figure 1. Using Eclipse to run GroovyTestSuite



In Figure 2, you see what happens when I click the Arguments tab and write the path to my script:

Figure 2. Specifying a Path to a Script in Eclipse



Running a favorite JUnit Groovy script really is as easy as locating the corresponding run configuration within Eclipse.

Testing with Ant and Maven

The beauty of a framework like JUnit is that it can run an entire suite of tests as part of a build without requiring human intervention. As more and more people add test cases to the code base, the overall test suite grows, serving as an excellent regression platform. What's more, build frameworks like Ant and Maven have added reporting features that summarize a JUnit batch run.

The easiest way to incorporate a host of Groovy test cases into a build is to compile them into normal Java byte code and include them in the standard JUnit batch commands offered by Ant and Maven. Fortunately, Groovy offers an Ant tag that incorporates compiling Groovy scripts into byte code, so the process of transforming scripts into functional byte code couldn't be any more straightforward. For example, if you happen to be utilizing Maven for builds, you simply need to add two new goals to your maven.xml file, two new dependencies in your project.xml file, one simple flag in your build.properties file, and you're set to go.

I'll start by updating my maven.xml file with the new goal to compile the example scripts, as shown in Listing 8:

Listing 8. Sample maven.xml file defining a Groovyc goal

```
<goal name="run-groovyc" prereqs="java:compile,test:compile">
  <path id="groovy.classpath">
    <pathelement path="${maven.build.dest}"/>
    <pathelement path="target/classes"/>
    <pathelement path="target/test-classes"/>
    <path refid="maven.dependency.classpath"/>
  </path>

  <taskdef name="groovyc" classname="org.codehaus.groovy.ant.Groovyc">
    <classpath refid="groovy.classpath"/>
  </taskdef>

  <groovyc destdir="${basedir}/target/test-classes" srcdir="${basedir}/test/groovy"
    listfiles="true">
    <classpath refid="groovy.classpath"/>
  </groovyc>
</goal>
```

A few things are happening in the above code. First, I've defined a new goal named `run-groovyc`. This goal has two prerequisites, `java:compile`, which compiles the example source code and `test:compile`, which compiles the any normal Java-JUnit classes. Next, I've created a classpath with the `<path>` tag. In this case, the classpath incorporates the build directory (where the compiled source resides) and all its associated dependencies (that is, JAR files). Next, I've defined the `groovyc` task with the `<taskdef>` Ant tag.

Notice, also, how I've told Maven where to find the class `org.codehaus.groovy.ant.Groovyc` in the classpath. In the last lines of the example I've defined a `<groovyc>` tag that compiles all Groovy scripts found in the `test/groovy` directory and places the resulting `.class` files in the `target/test-classes` directory.

Some important details

In order to compile Groovy scripts and run their resulting byte code, I would have to define two new dependencies (*groovy* and *asm*) via the `project.xml` file, as shown in Listing 9:

Listing 9. New dependencies for a project.xml file

```
<dependency>
  <groupId>groovy</groupId>
  <id>groovy</id>
  <version>1.0-beta-6</version>
</dependency>

<dependency>
  <groupId>asm</groupId>
  <id>asm</id>
  <version>1.4.1</version>
```

```
</dependency>
```

Once the scripts are compiled to normal Java bytecode they can be run by any standard JUnit runner. Because Ant and Maven have a JUnit runner tag, the next step is to have JUnit pick up the newly compiled Groovy scripts. And because the Maven JUnit runner uses pattern matching to find test suites to run, I need to add a special flag in the build.properties file, as shown in Listing 10, which tells Maven to search classes instead of .java files:

Listing 10. A Maven project build.properties file

```
maven.test.search.classdir = true
```

Lastly, I define a test goal in the maven.xml file shown in Listing 11. Doing so ensures the Groovy scripts will be compiled with the new run-groovyc goal before any unit tests are run.

Listing 11. A new goal for maven.xml

```
<goal name="test">
  <attainGoal name="run-groovyc"/>
  <attainGoal name="test:test"/>
</goal>
```

Last but not least ...

With the two new goals defined (one to compile scripts and the other to run the combined Java and Groovy JUnit tests), the only thing left to do is run them and verify that everything is working!

In Listing 12, you see what happens when I run Maven and pass in the test goal, which first attains the run-groovyc goal (which also happens to attain the java:compile and test:compile goals) then attains the standard out-of-the-box Maven test:test goal. Notice how the test:test goal picks up both of the newly created Groovy scripts -- or in this case, the newly *compiled* Groovy scripts -- and the normal Java JUnit test.

Listing 12. Running the new goal

```
$ ./maven test

test:
java:compile:
[echo] Compiling to /home/aglover/dev/target/classes
[javac] Compiling 15 source files to /home/aglover/dev/target/classes

test:compile:
```

```
[javac] Compiling 4 source files to /home/aglover/dev/target/test-classes
run-groovyc:
[groovyc] Compiling 2 source files to /home/aglover/dev/target/test-classes
[groovyc] /home/aglover/dev/test/groovy/test/RegexFilterTest.groovy
[groovyc] /home/aglover/dev/test/groovy/test/SimpleFilterTest.groovy
test:test:
[junit] Running test.RegexFilterTest
[junit] Tests run: 1, Failures: 0, Errors: 0, Time elapsed: 0.656 sec
[junit] Running test.SimpleFilterTest
[junit] Tests run: 1, Failures: 0, Errors: 0, Time elapsed: 0.609 sec
[junit] Running test.SimplePackageFilterTest
[junit] Tests run: 1, Failures: 0, Errors: 0, Time elapsed: 0.578 sec
BUILD SUCCESSFUL
Total time: 42 seconds
Finished at: Tue Sep 21 17:37:08 EDT 2004
```

For more information on using Groovy and Maven together, check out the [GMaven project](#).

Reviewing today's lesson

In this first installment of *Practically Groovy*, you've learned about one of the most practical applications of this exciting new scripting language. For a growing number of developers, unit testing is an essential part of the development process; and with Groovy and JUnit, unit testing Java code is a snap.

Groovy's simple syntax and built-in agility make it an excellent platform for quickly writing effective JUnit tests and incorporating them into an automated build. For a code-quality addict like myself, this combination drastically reduces the *heebie-jeebies* and allows me to get to what I like doing best: writing bullet-proof software. Quickly.

Because this is a new series, you're strongly encouraged to help steer its progress. If there's something you want to know about Groovy, [drop me a line](#) and let me know! In the meantime, I hope you'll tune in for the next installment in which I'll be talking about Ant scripting with Groovy.

Downloads

Description	Name	Size	Download method
Sample code	j-pg11094.zip	4KB	HTTP

[Information about download methods](#)

About the authors

Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the [easyb](#) Behavior-Driven Development (BDD) framework and is the co-author of three books: [Continuous Integration](#), [Groovy in Action](#), and [Java Testing Patterns](#). He teaches a wide variety of Groovy-, Grails-, and testing-related classes at [ThirstyHead.com](#). You can keep up with Andy at [thediscoblog.com](#) where he routinely blogs about software development.

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).